B.Sc. in Computer Science and Engineering Thesis

Streaming Online Principal Components Analysis

Submitted by

Mohammad Imam Hossain
201105031

Supervised by

Dr. Muhammad Abdullah Adnan

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

February 2017

CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, "Streaming Online Principal Components Analysis", is the outcome of the investigation and research carried out by us under the supervision of Dr. Muhammad Abdullah Adnan.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

_____

Mohammad Imam Hossain
201105031

CERTIFICATION

This thesis titled, **"Streaming Online Principal Components Analysis"**, submitted by the group as mentioned below has been accepted as satisfactory in partial fulfillment of the requirements for the degree B.Sc. in Computer Science and Engineering in February 2017.

Group Members:

Mohammad Imam Hossain

Supervisor:

Dr. Muhammad Abdullah Adnan
Assistant Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

ACKNOWLEDGEMENT

Foremost, I would like to express my sincere gratitude to my advisor Prof. Dr. Muhammad Abdullah Adnan for the continuous support of my research, for his patience, motivation, enthusiasm, and immense knowledge. His guidance helped me in all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for my undergraduate research and thesis.

A special thanks to my family. Words cannot express how grateful I am to my parents for all of the sacrifices that youve made on my behalf. Your prayer for me was what sustained me thus far. I would also like to thank all of my friends, thesis mates who supported me in writing, and motivated me to strive towards my goal.

Lastly, I offer my regards and blessings to all of those who supported me in any respect during the completion of the thesis.

Dhaka
February 2017

Mohammad Imam Hossain

Contents

CANDIDATES' DECLARATION i

CERTIFICATION ii

ACKNOWLEDGEMENT iii

List of Figures vii

List of Algorithms viii

ABSTRACT ix

1 Introduction 1

1.1 PCA using Eigenvalue Decomposition . . . 2
1.2 PCA using SVD . . . 3
1.3 Online PCA . . . 3
1.4 Our Contribution: Streaming Online PCA . . . 5
1.5 Outline of the Thesis . . . 5

2 Research Background and Motivation 7

2.1 Motivation . . . 7
2.2 Literature Review . . . 7
2.2.1 Simple and Deterministic Matrix Sketching . . . 8
2.2.2 Random Projection Method . . . 8
2.2.3 Regret Minimization Approach . . . 8
2.2.4 Block-Stochastic Power Method . . . 9
2.2.5 Online PCA using Frobenius Norm . . . 10
2.2.6 Online PCA with Spectral Bounds . . . 10
2.3 The Chapter at a Glance . . . 11

3 Background Study 12

3.1 Different Types of Norms: . . . 12
3.1.1 Vector Norms . . . 12
3.1.2 Matrix Norms . . . 13

3.2 Eigenvector and Eigenvalue 14
3.3 Singular Value Decomposition 15
3.4 Eigendecomposition 15
3.5 Variance and Covariance 16
3.6 The Chapter at a Glance 17

4 Streaming Online PCA Algorithm **18**
4.1 Introduction 18
4.2 Flow Chart 18
4.3 The Algorithm 19
4.4 Notations 20
4.5 Findings 20
4.5.1 Observations 20
4.5.2 Relation between Isometry Matrix and Orthogonal Matrix 22
4.5.3 Time and Space Complexity 22
4.5.4 Bound on Output Dimension 23
4.6 The Chapter at a Glance 24

5 Streaming Online PCA with Fixed Output Dimension **25**
5.1 Introduction 25
5.2 The Algorithm 25
5.3 Notations 26
5.4 Flow Chart 26
5.5 Findings 27
5.5.1 Observations 27
5.5.2 Time and Space Complexity 28
5.6 The Chapter at a Glance 29

6 Experimental Results **30**
6.1 Experimental Setup 30
6.2 Comparing Outputs between Different Algorithms 30
6.2.1 Comparison based on Reconstruction Error 30
6.2.2 Comparison based on Reduced Output Dimension 31
6.3 Streaming Online PCA Algorithm and Observations 32
6.3.1 Reconstruction Error vs Block Size Tuning 32
6.3.2 Change in Reconstruction Error with the Change in Fixed Error Bound . 33
6.4 Streaming Online PCA with Fixed Output Dimension and Observations 33
6.4.1 Changes in Reconstruction Error with the Change in Output Vectors Dimension 34
6.4.2 Changes in Reconstruction Error with the Change in Fixed Error Bound 34

6.4.3 Effect of Change in Buffer Size, b 35
6.5 The Chapter at a Glance 36

7 Conclusion **37**
7.1 Future Works 37

References **38**

A Codes **41**
A.1 Streaming Online PCA Algorithm 41
A.2 Streaming Online PCA with Fixed Output Dimension 42
A.3 Observation 1 Experimental Code 43
A.4 Observation 2 Experimental Code 44
A.5 Observation 5 Experimental Code 45
A.6 Observation 6 Experimental Code 45
A.7 Observation 7 Experimental Code 46

List of Figures

1.1 Offline PCA 1
1.2 Finding Principal Vector with Greatest Variance 2
1.3 PCA using SVD 3
1.4 Online PCA 4
1.5 Streaming Online PCA 5

3.1 Interpretation of SVD 15
3.2 Singular Value Decomposition 15

4.1 Flow chart of Algorithm 1 19

5.1 Flow chart of Algorithm 2 27

6.1 Reconstruction error vs Delta 31
6.2 output dimension l vs Delta 32
6.3 Impact on reconstruction error with the change in block size 32
6.4 Relation between reconstruction error and fixed error bound 33
6.5 Relation between reconstruction error and output dimension as described in Observation 5 34
6.6 Observation 6 experimental results 35
6.7 Observation 7 experimental results 35

List of Algorithms

1 Streaming Online PCA algorithm 20
2 Streaming Online PCA algorithm with fixed output dimension 26

ABSTRACT

Principal Component Analysis (PCA) is a fundamental tool used in dimensionality reduction, data analysis, clustering, machine learning, classification, information retrieval applications and many more advanced scientific research fields. It is important for computer science, statistics, engineering and many scientific computing fields. It is mainly used to obtain a lower dimensional representation of a high dimensional data that still captures as much of the original data as possible. Because of its lower dimension, it is useful for reducing storage and computational costs.

This thesis considers the streaming version of online PCA problem. In the standard PCA, the input is a set of vectors X each of dimension d, $X = [x_1, x_2, \ldots, x_n]\, \epsilon \mathbb{R}^{d \times n}$ and a target dimension k where $k < d$. The output is a set of vectors Y each of dimension k, $Y = [y_1, y_2, \ldots, y_n]\, \epsilon \mathbb{R}^{k \times n}$ that minimizes the reconstruction error: $\|X - \varphi Y\|^2$. Here, $\varphi \epsilon \mathbb{R}^{d \times k}$ is an isometric projection matrix. The best optimum value of this reconstruction error can be obtained in standard offline PCA algorithm. In the online version of PCA problem, each of the input vectors x_t is given input to the system one by one and the system generates the corresponding output vectors y_t at the same time before getting the next input. If l denotes the output dimension in online PCA, then $k \leq l \leq d$.

In this thesis, we propose two online PCA algorithms. The first one simply uses the streaming buffer block concept where it receives a fixed error bound Δ and a block size b ensuring all the time that the error will not exceed Δ. The second algorithm also considers the streaming buffer concept and takes an extra parameter l as input that represents the target dimension of output vectors. Although this type of streaming blocked concept has been applied in the offline PCA case in the past but to the best of our knowledge, this thesis is the first to consider this kind of setting for online PCA problem.

Chapter 1

Introduction

Principal Component Analysis (PCA) is a statistical procedure that is used for dimensionality reduction, factor analysis, correlation analysis, clustering classification and many more advanced scientific computing and research fields. PCA is a linear orthogonal transformation that transforms the data to a new coordinate system such that the maximum variance by any projection of the data comes to lie on the first coordinate known as the first principal component, the second maximum variance on the second coordinate and so on. So, Principal Component Analysis computes the most meaningful basis to express our data and provides a lower dimensional representation. Here, basis represents a set of vectors in a linear combination that are linearly independent and can represent every vector (they form a coordinate system) [1]. It can be defined as optimizing the dimension of input vectors with minimum reconstruction error. In standard PCA problem [2, 3], the input is a set of vectors $x_1, x_2, \ldots, x_n \epsilon \mathbb{R}^d$ of dimension d and a target dimension of output vectors k $(k \leq d)$. The solution is a set of output vectors $y_1, y_2, \ldots, y_n \epsilon \mathbb{R}^k$ such that the reconstruction error is minimized. The reconstruction error can be defined in two forms, the Frobenius form : $\|X - \varphi Y\|_F^2$ and the Spectral form: $\|X - \varphi Y\|_2^2$ where $\varphi \epsilon \mathbb{R}^{d \times k}$ is the isometric projection matrix.

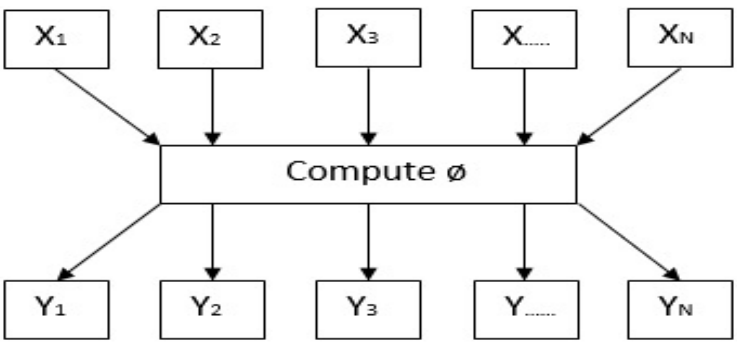

Figure 1.1: Offline PCA

Principal components can be represented as the principal axis of a n dimentional ellipsoid.The main objective of PCA is to fit the data to the n-dimensional ellipsoid. Small axis of the ellipsoid represents small variance along that axis and by ignoring that axis and its corresponding principal component, we lose only a small amount of information that is not much significant [4].

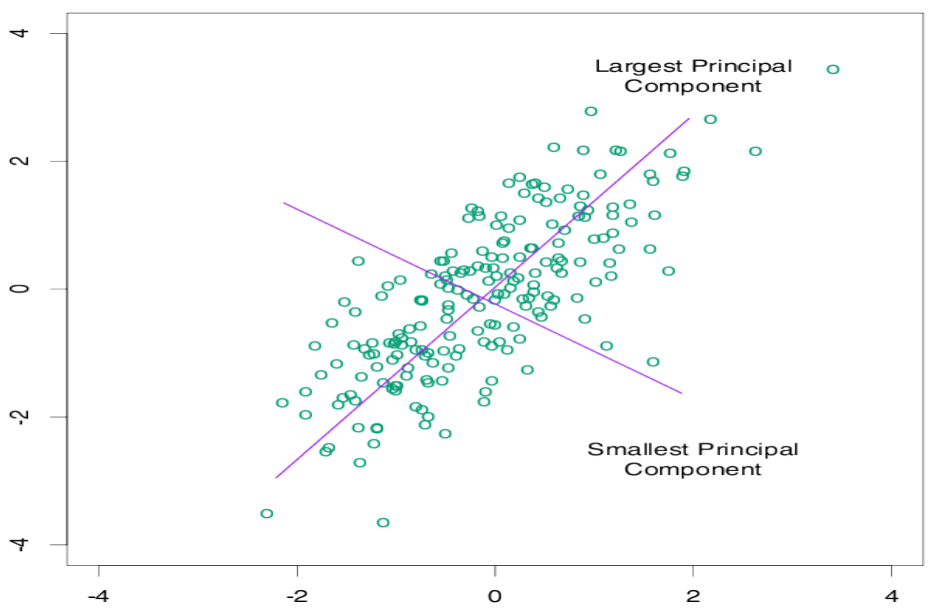

Figure 1.2: Finding Principal Vector with Greatest Variance

1.1 PCA using Eigenvalue Decomposition

To find the axes of the ellipsoid, we must first subtract the mean of each variable from the dataset to center the data around the origin. Then, we compute the covariance matrix of the data, and calculate the eigenvalues and corresponding eigenvectors of this covariance matrix. Then, we must orthogonalize the set of eigenvectors, and normalize each to become unit vectors. Once this is done, each of the mutually orthogonal, unit eigenvectors can be interpreted as an axis of the ellipsoid fitted to the data. The proportion of the variance that each eigenvector represents can be calculated by dividing the eigenvalue corresponding to that eigenvector by the sum of all eigenvalues. let, $X_{p\times n}$ represents the centered input matrix of dimension $p \times n$. Then the covariance matrix is

$$C = \frac{XX^T}{n-1} = ULU^T$$

where U is a matrix of eigenvectors (each column is an eigenvector) and L is a diagonal matrix with eigenvalues λ_i in the decreasing order on the diagonal. The eigenvectors are called principal axes or principal direction of data. Projections of the data points on the principal axes are called principal components.The j-th principal component is given by j-th row of U^TX and the coordinates of i-th data point in the new Principal Component space are given by the i-th column of U^TX.

1.2 PCA using SVD

The best way to solve this standard offline PCA problem is to center the input matrix X and perform the Singular Value Decomposition (SVD) [5, 6] on it. Let the SVD of X is USV^T, then the best isometric projection will be $\varphi = U_k$, where $U_k \epsilon \mathbb{R}^{d \times k}$ consists of k left singular vectors of U corresponding to the top k singular values of S. The corresponding output vector will be $y_t = U_k^T x_t$, where x_t is the input vector at time t and y_t is the output vector at time t. As we need the whole input matrix X for generating the best projection plane φ, so for this offline PCA case we get the best optimal reconstruction error and the best isometric projection matrix. The following image shows the process:

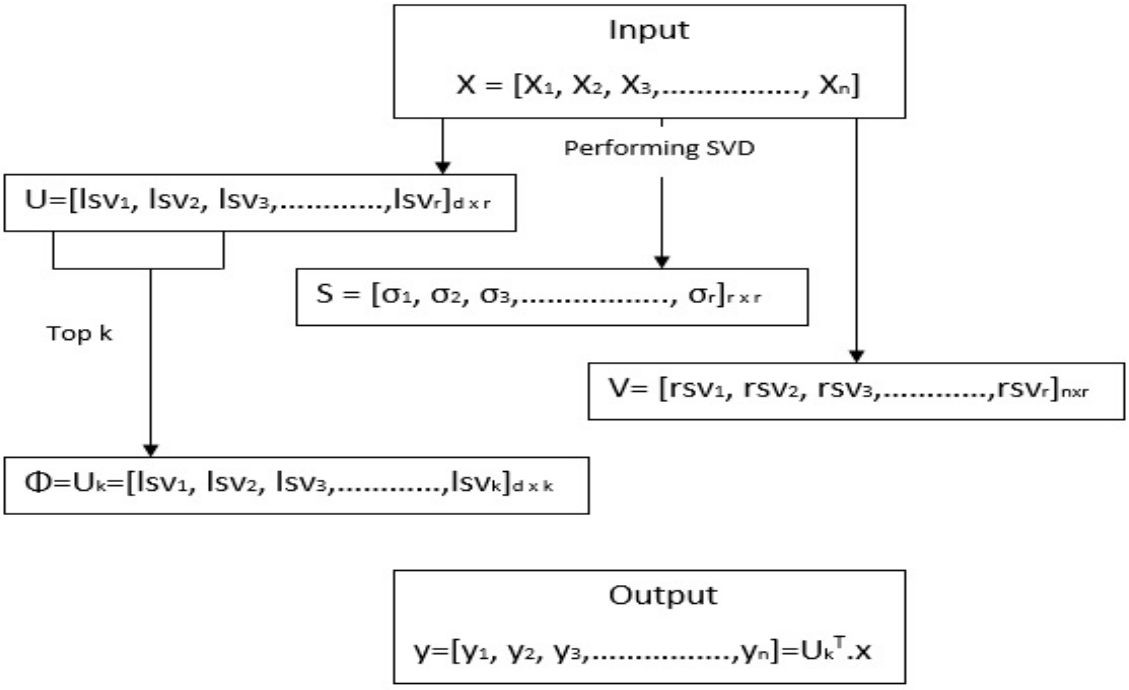

Figure 1.3: PCA using SVD

1.3 Online PCA

Already we have discussed about offlien standard PCA and how it finds its principal vectors and components. It generates all the reduced dimensional output vectors at the same time.

In online PCA problem, each of the input vectors x_t of dimension d is passed into the algorithm one after another and before passing the next one x_{t+1} the algorithm must generate the corresponding output vector y_t of reduced dimension l. So it generates reduced dimensional output vectors one after another.

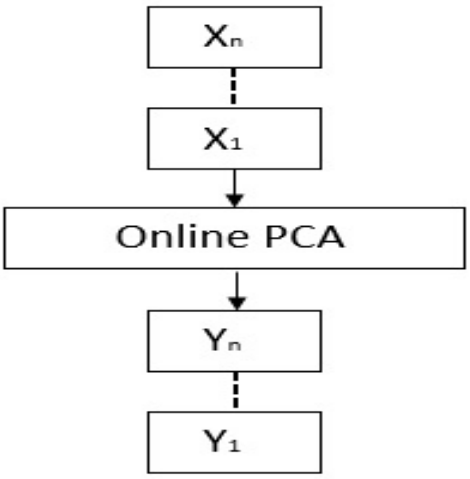

Figure 1.4: Online PCA

So it is obvious for the online PCA that the reconstruction error and the reduced dimension of output vectors won't be as optimal as the offline standard PCA. It is a great challenge to establish an online PCA algorithm that will simultaneously reduce the dimension of output vectors and minimize the reconstruction error. From the definition of Frobenius norm we know that,

$$\|A\|_F^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2 = \sum_{i=1}^{R} \sigma_i^2 \tag{1.1}$$

where m, n represents the row and column number of matrix A, R represents the rank of matrix A and σ_i represents the i-th top singular value of A. And the spectral norm is,

$$\|A\|_2^2 = \sigma_{max}^2 (A) \tag{1.2}$$

In the standard offline PCA, the best projection plane is not dependent on whether or not we consider the reconstruction error in Frobenius or Spectral perspective [7]. But for online PCA, the projection plane varies based on our consideration of reconstruction error in Frobenius norm or Spectral norm form. As we can't build the projection plane based on the whole input vectors X for online PCA case and the selection of best principal components depends on the fixed error bound, so the projection plane varies based on Frobenius or Spectral form of reconstruction error consideration.

Already there exists some online PCA algorithms considering both Frobenius and Spectral norm form of reconstruction error. But our model is different as we solve the online PCA problem with respect to Spectral norm with streaming buffer concept to choose the best projection plane.

1.4 Our Contribution: Streaming Online PCA

Actually our algorithm is inspired by the paper [8]. We consider a history (Streaming buffer) of size equals to block size b and each time we choose the best principal component based on the history. We consider Spectral norm error (as reconstruction error) for the algorithms proposed in this thesis. As Frobenius norm doesn't hold for input matrix containing heavy tail of singular values and Spectral norm low rank approximation error is more stronger than the Frobenius norm error [9].

Our first algorithm considers the fixed error bound model, where a fixed error Δ and a fixed block size b is given input to the algorithm. Always the algorithm ensures that the spectral error at time t, $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 < \Delta$. Also we provide an upper bound on the dimension of output vector l that is better than the output vector's dimension range shown in the paper [8].

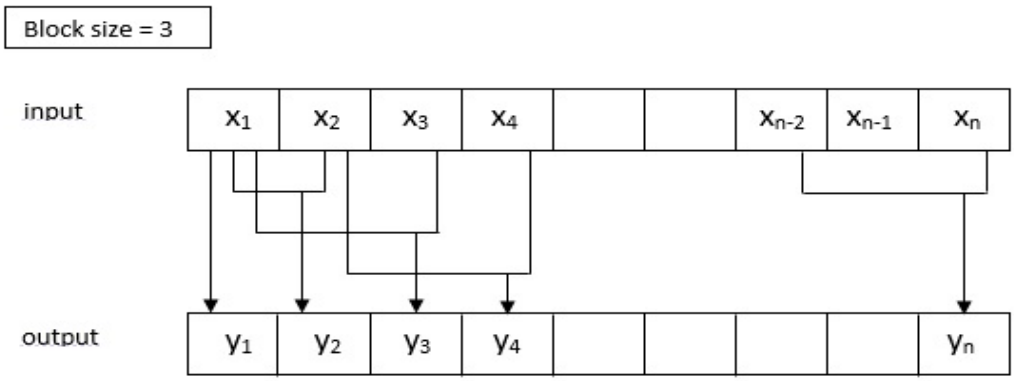

Figure 1.5: Streaming Online PCA

The second algorithm considers the fixed output dimension model that takes the fixed error bound Δ, streaming buffer size b and the reduced output dimension l as input. The main speciality of this second algorithm is that it takes the desired reduced output vectors dimension (l) as input and at the end of execution of this algorithm the projection φ contains the best l principal components. The reduced dimension of output vectors and reconstruction error provided by both of our algorithms are better than any other online PCA algorithms.

1.5 Outline of the Thesis

Chapter 2 describes the previous contributions in this PCA field. It describes different algorithms regarding PCA. It also describes the most recent works in this field. The limitations of this field are also described in this chapter. It also describes the former works based on frobenius norm, spectral norm, different sketching techniques, different streaming PCA algorithms

in the literature.

Chapter 3 gives us the background knowledge that is required for complete understanding of this thesis and it's mathematical, theoretical analyses. It also provides a brief description of different types of vector, matrix norms that is required for computing reconstruction error. It also explain the singular value decomposition and eigen-decomposition methods to understand how the principal components analysis works. It also illustrates the variance and covariance methods. Different types of co-variance sketching methods are also illustrates in this chapter.

In Chapter 4, we describe our first algorithm that is based on streaming buffer block concept. The algorithm uses a buffer to handle the streaming online data. It also provides a bound on reduced output dimension. The algorithm always guarantees that the reconstruction error is within a fixed error bound.

Chapter 5 illustrates our second algorithm that is streaming online pca with fixed output dimension. This algorithm takes output vector dimension as input and keeps track of the best l principal components obtained so far at that time and outputs the reduced vector based on that l principal components found at that time.

In chapter 6, we provide the experimental results that supports our algorithms. It also compares our proposed algorithm with other online PCA algorithms. Experimentally it also shows the results of different observations obtained from our algorithms.

The concluding chapter gives some directions of future research. It also outlines the challenges in these fields and scope of work in these fields.

Chapter 2

Research Background and Motivation

2.1 Motivation

Principal component analysis (PCA) is a statistical and analytical technique used to find out the direction of highest variation and bring out strong patterns in a dataset. It's often used to make data easy to explore and visualize. It is mainly used for dimensionality reduction,classification, clustering and many scientific fields. Offline standard PCA provides the best optimal principal components as it considers all the input vectors at the same time. So it provides the best optimum reconstruction error.

In online PCA, the algorithm gets one vector at a time. It predicts the best principal components at that time before considering the next input vector. Different types of conceptual, space efficient, time efficient algorithms exist in the literature. But these algorithms consider covariance sketching and incremental techniques that leads our streaming buffer block concept in online PCA.

In our streaming online PCA we are considering a streaming buffer to keep track of the last buffer length sized input vectors. Then based on the buffer history we will predict the best principal component obtainable upto current time period. Our algorithms also maintains a fixed error limit to limit the reconstruction error.

2.2 Literature Review

Some papers have already provided different versions of algorithms to solve online PCA problem. But they also have some limitations. Some of the major contributions in this fields are described below:

2.2.1 Simple and Deterministic Matrix Sketching

In order to obtain a smaller sized estimate of a matrix we have to sketch the matrix using sketching algorithms. There exists several matrix sketching approaches in the literature. The first approach generates sparse matrix that can be stored efficiently and multiplied faster with other matrices [10–12]. The second approach uses the machanism to combine randomly rows of the matrix [13–15]. The Frequent Direction matrix sketching algorithm is presented in the paper citeliberty2013simple. The third approach used column subset selection method [16,17] . The fourth approach is the frequent direction matrix approximation that is an extension of item frequency approximation problem. For any matrix $A\epsilon\mathbb{R}^{n\times m}$, this frequent direction algorithm generates a sketch matrix $B\epsilon\mathbb{R}^{l\times m}$. This sketching technique also provides an upper bound of sketching error

$$\left\|A^TA - B^TB\right\| \leq 2\left\|A\right\|_F^2/2$$

2.2.2 Random Projection Method

The algorithms presented in the paper [15] constructs projection matrix independently and randomly from the data. The main feature is to remove data dependency and provide significant reduction in time for matrix multiplication and singular value decomposition and linear regression. Before this random projection method, the rank k approximation of input matrix $A\epsilon\mathbb{R}^{m\times n}$, A_k is bounded through projection and several sampling based results of the form:

$$\left\|A - \hat{A}_k\right\|_F \leq \left\|A - A_k\right\|_F + \epsilon^t\left\|A\right\|_F$$

But this random projection method provides the most efficient relative error approximation that requires fewer passes 2 instead of $O\left(k.logk\right)$ and faster in k i.e. $r = O\left(\frac{k}{\epsilon}\right)$ instead of $r = O\left(\frac{k}{\epsilon} + k^2.logk\right)$

$$\left\|A - \hat{A}_k\right\|_F \leq (1+\epsilon)\left\|A - A_k\right\|_F$$

The main limitation of this method is the reconstruction matrix that is not an isometry in general.

2.2.3 Regret Minimization Approach

This paper named *Online PCA with Optimal Regrets* [18] based on regret minimization provides regret bounds for Gradient Descent(based on regularization of squared euclidean distance, additive update algorithm) and Matrix Exponentiated Gradient(based on relative entropy as regularization, multiplicative update algorithm) online algorithms. It also provides bound for both the sparse and dense setting. The paper considers online centered PCA. In every trial, the algorithm selects k dimensional subspace based on the previous history $(x_1, x_2, ..., x_n)$ and provides

an online algorithm such that the cumulative compression loss is close to the best rank k projections (Compression Loss:$\|x_t - P_t x_t\|_2^2 = tr\left((I - P_t)\, x_t x_t^T\right)$ .Here, regret means the difference between the cumulative loss of the algorithm and the best offline compression loss. This method does not explain how to decompose the projection matrix $p_t = U_t U_t^T$ that is the main limitation of this algorithm for dimension reduction.

The bounds for MEG and for both sparse and dense settings provided by this setting are shown below:

For sparse instances of size T,

$$regret_{LossMEG} \leq m\sqrt{\frac{2T}{n} log\frac{n}{m}} + mlog\frac{n}{m} \leq \sqrt{\frac{2kmT}{n}} + k$$
$$regret_{GainMEG} \leq \sqrt{2kTlog\frac{n}{k}}$$

For dense instance sequence of length T,

$$regret_{LossMEG} \leq m\sqrt{2Tlog\frac{n}{m}} + mlog\frac{n}{m}$$
$$regret_{GainMEG} \leq k\sqrt{2Tlog\frac{n}{k}}$$

where, rank of projection matrix p_t is k
complementary projection matrix $(I - P_t)$ has rank $m = n - k$
and n = dimension of the data.

2.2.4 Block-Stochastic Power Method

In the paper [19], Blocked-stochastic power method is used. The paper uses the streaming block concept for offline standard PCA case. The main objective of that paper is to reduce the variance by averaging out noises by using modified stochastic power method. The algorithms presented in that paper achieve best lower bounds on both memory $O\,(kp)$ and sample complexity of $O\,(plogp)$. The algorithm assumes a probabilistic generative model and at each time step t, it receives a point $x_t \epsilon \mathbb{R}^p$, where

$$x_t = Az_t + w_t$$

Here $A\epsilon\mathbb{R}^{p\times k}$ is the projection matrix, $z_t\epsilon\mathbb{R}^{k\times 1}$ represents the multivariate normal variable

$$z_t \sim N\left(0_{k\times 1}, I_{k\times k}\right)$$

and the noise vector $w_t \epsilon \mathbb{R}^{p\times 1}$ is also sampled from multivariate normal random variable.

$$w_t \sim N\left(0_{p\times 1}, \sigma^2 I_{p\times p}\right)$$

The algorithm uses block-wise method where all the n-sampled inputs are divided into several blocks. In the $\tau + 1$-st block, the algorithm computes

$$s_{\tau+1} = \left(\frac{1}{B}\sum_{t=B_{\tau+1}}^{B(\tau+1)} x_t x_t^T\right) q_\tau$$

Then it finds $q_{\tau+1}$ from q_τ using

$$q_{\tau+1} = s_{\tau+1} / \left\|s_{\tau+1}\right\|_2$$

Here, the algorithm also guarantees that $\left\|q_\tau - u\right\|_2 \leq \epsilon$ where, $u \epsilon \mathbb{R}^p$ represents the principal component
τ represents the block number,
B represents the size of the Block.

2.2.5 Online PCA using Frobenius Norm

Also two algorithms have been suggested in the paper [20] that have used Frobenius norm as a measure of reconstruction error. But the first algorithm presented in that paper requires to input $\|X\|_F^2$. This requires prior knowledge of the Frobenius norm of the whole input vectors matrix X. Then the paper suggests a 2nd algorithm that doesn't require to take input $\|X\|_F^2$ but instead this algorithm uses Frequent Direction Sketching [21] method. This sketching adds extra error to the algorithm.

2.2.6 Online PCA with Spectral Bounds

The algorithms suggested in the paper [8] have also some limitations. The first algorithm *Fixed Error: Conceptual Algorithm* has used incremental block size. As a result, at the end of iteration the incremental block becomes the input matrix X. So it sacrifices the main purpose of online PCA. Also the output dimension of each vector are different at different time as it depends on the fixed error bound Δ. The second algorithm *Fixed Error: Space Efficient Algorithm* has used sketching instead of using incremental block $X_{1:t}$. As a result, it maintains the main objective of online PCA, but adds extra error ρ due to using covariance matrix sketching. The output dimension also varies in this case. Both of these two algorithms also require to input fixed error bound Δ. If we don't have some prior knowledge about the error then this will be a real

problem. The third one *Adaptive Error: Time Efficient Algorithm* automatically handles the fixed error bound Δ. But it also uses the covariance sketching [21–23] technique, that adds extra error ρ to the algorithm.

2.3 The Chapter at a Glance

This chapter describes different online PCA algorithms and their scope in the literature. This chapter also explains their workign mechanism and their limitations. Based on these limitations the following chapters describe two online PCA algorithms. Actually, there exists no online PCA algorithm that considers both streaming buffer concept with spectral bounds. This thesis presents algorithms that consider both streaming buffer block with spectral bounds. Besides the algorithms also improve both the dimension of output vector and reconstruction error.

Chapter 3

Background Study

Principal Components Analysis is an algebraic technique used for different purposes. In order to continue this thesis work we need some background mathematical knowledge that will be helpful. The mandatory fields that are required for this thesis are described below:

3.1 Different Types of Norms:

To find out the distance between different vectors and matrices, we need to calculate the norms and comparing them. Mainly used vector and matrix norms are mentioned below:

3.1.1 Vector Norms

The norm of a vector $x\epsilon V$ in vector space V is a real non-negative value representing intuitively the length, size, or magnitude of the vector [24]. Specifically, the norm of $x\epsilon V$ must satisfy the following three conditions:

1. $\|x\| \geq 0, \|x\| = 0$ iff $x = 0$
2. $\|ax\| = |a| \, \|x\| \, , a\epsilon\mathbb{R} or \mathbb{C}$
3. $\|x + y\| \leq \|x\| + \|y\|$

Here are some forms of common vector norms:

1. If the vector $x = x\epsilon\mathbb{R}$ is a real number, then its norm is simply its absolute value $\|x\| = |x|$.

2. If $x = [x_1, x_2, x_3, ..., x_n]^T$ is a vector in an n-D vector space $V = \mathbb{R}^n$ or $V = \mathbb{C}^n$, then we can use the p norms defined as:

$$\|x_p\| = \left(\sum_1^n |x_i^p|\right)^{\frac{1}{p}}$$

   (a) if p=1,

$$\|x\|_1 = \sum_1^n |x_i|$$

   (b) if p=2,

$$\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_1^n |x_i|^2}$$

   (c) if $p = \infty$,

$$\|x\|_\infty = max\{|x_1|, |x_2|, |x_3|, ..., |x_n|\}$$

3.1.2 Matrix Norms

An $m \times n$ matrix A can be considered as a particular kind of vector $X = A\epsilon\mathbb{R}^{m,n}$ [25]and its norm is any function that maps A to a real no. $\|A\|$ that satisfies the following properties:

1. $\|A\| \geq 0, \|A\| = 0$ iff $A = 0$
2. $\|aA\| = |a| \|A\|$
3. $\|A + B\| \leq \|A\| + \|B\|$

Some commonly used matrix norms are given below:

1. Element-wise norms:

$$\|A\|_p = \left\{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^p\right\}^{\frac{1}{p}}$$

   where,

   (a) $\|A\|_1 = \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|$
   (b) $\|A\|_\infty = max\{|a_{ij}|; 1 \leq i \leq m, 1 \leq j \leq n\}$
   (c) $\|A\|_2 = \|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2} = \sqrt{tr\,(A^T A)} = \sqrt{\sum_{i=1}^R \lambda_i} = \sqrt{\sum_{i=1}^R \sigma_i^2}$

   here, $R \leq min\,(m, n)$ is the rank of A
   $\lambda_i = \sigma_i^2$ is the i-th non-zero eigenvalue of $\left(A^T A\right)$
   $\sigma_i = \sqrt{\lambda_i}$ is the i-th singular value of A

2. Induced/Operator norms:

$$\|A\| = sup_{\|x\|=1}\{\|Ax\|\} = sup_{x\neq 0}\left\{\frac{\|Ax\|}{\|x\|}\right\}$$

   (a) $\|A\|_1 = max_{1\leq j\leq n}\sum_{i=1}^{m}|a_{ij}| = sup_{\|x\|_p=1}\{\|Ax\|_1\}$
   (b) $\|A\|_\infty = max_{i\leq i\leq m}\sum_{j=1}^{n}|a_{ij}| = sup_{\|x\|_\infty=1}\{\|Ax\|_\infty\}$
   (c) $\|A\|_2 = \sigma_{max}(A) = \sqrt{\lambda_{max}(A^TA)} = sup_{\|x\|_2=1}\{\|Ax\|_2\}$

   here, $\lambda_{max}\left(A^TA\right) = max^m$ eigenvalue of $\left(A^TA\right)$
   $\sigma_{max}(A) = \sqrt{\lambda_{max}(A^TA)} = max^m$ singular value of A

3.2 Eigenvector and Eigenvalue

A scalar λ is called an eigenvalue of the $n \times n$ matrix A if there is a nontrivial solution x of $Ax = \lambda x$. Such an x is called an eigenvector corresponding to the eigenvalue .
Suppose that A is the standard matrix for a linear transformation $T : R^n \to R^n$. Then if $Ax = \lambda x$, then $T(x) = \lambda x$. This means that if x is an eigenvector of A, then the image of x under the transformation T is a scalar multiple of x and the scalar involved is the corresponding eigenvalue . In other words, the image of x is parallel to x. Retrieved from [26]. In standard form:

$$(A - \lambda I)x = 0$$

here $(A - \lambda I)$ is not invertible i.e. $(A - \lambda I)$ is singular.
So

$$|A - \lambda I| = 0$$
$$\Rightarrow (\lambda - \lambda_1)(\lambda - \lambda_2)(\lambda - \lambda_3)...(\lambda - \lambda_n) = 0$$

Properties:

1. $tr(A) = \sum_{i=1}^{n} A_{ij} = \sum_{i=1}^{n}\lambda_i$
2. $det(A) = \prod_{i=1}^{n}\lambda_i$
3. if $eigs(A) = (\lambda_1, \lambda_2, \lambda_3, ..., \lambda_n)$,
   then $eigs\left(A^k\right) = \left(\lambda_1^k, \lambda_2^k, \lambda_3^k, ..., \lambda_n^k\right)$
   and $eigs\left(A^{-1}\right) = \left(\lambda_1^{-1}, \lambda_2^{-1}, \lambda_3^{-1}, ..., \lambda_n^{-1}\right)$

3.3 Singular Value Decomposition

The singular value decomposition (SVD) factorizes a linear operator $A : R^n \to R^m$ into three simpler linear operators:

1. Projection $z = V^T x$ into an r-dimensional space, where r is the rank of A
2. Element-wise multiplication with r singular values $_i$, i.e., $z' = Sz$
3. Transformation $y = Uz'$ to the m-dimensional output space

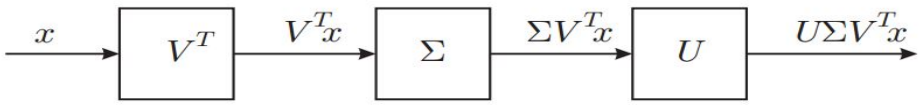

Figure 3.1: Interpretation of SVD

Combining these statements, A can be re-written as

$$A = USV^T$$

with U an $m \times r$ orthonormal matrix spanning As column space $im\,(A)$, S an $r \times r$ diagonal matrix of singular values, and V an $n \times r$ orthonormal matrix spanning As row space $im\left(A^T\right)$.

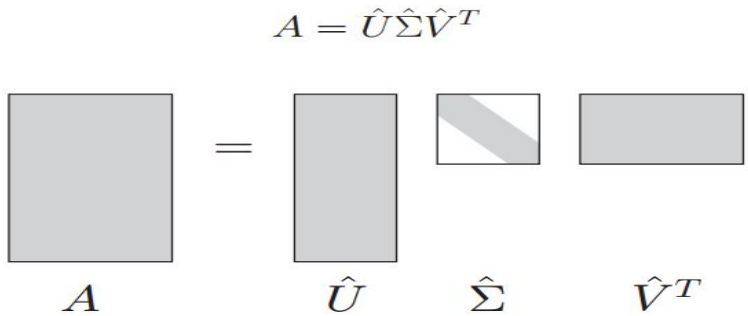

Figure 3.2: Singular Value Decomposition

For more details visit [27, 28].

3.4 Eigendecomposition

The eigenvalue decomposition applies to mappings from R^n to itself, i.e. a linear operator $A : R^n \to R^n$ described by a square matrix. An eigenvector e of A is a vector that is mapped to

a scaled version of itself, i.e., $Ae = \lambda e$, where λ is the corresponding eigenvalue. For a matrix A of rank r, we can group the r non-zero eigenvalues in an $r \times r$ diagonal matrix Λ and their eigenvectors in an $n \times r$ matrix E, and we have

$$AE = E\Lambda$$

Furthermore, if A is full rank $(r = n)$ then A can be factorized as

$$A = E\Lambda E^{-1}$$

which is a diagonalization similar to the SVD. In fact, if and only if A is symmetric and positive definite (abbreviated SPD), we have that the SVD and the eigen-decomposition coincide

$$A = USU^T = E\Lambda E^1$$

with $U = E$ and $S = \Lambda$.
Given a non-square matrix $A = USV^T$, two matrices and their factorization are of special interest:

$$AA^T = US^2U^T = E\Lambda E^1$$

Thus, for these matrices the SVD on the original matrix A can be used to compute their SVD. And since these matrices are by definition SPD, this is also their eigen-decomposition, with eigenvalues $\Lambda = S^2$. Retrieved from [29].

3.5 Variance and Covariance

The expected value of a random variable X gives a measure of the center of location of the distribution of that random variable. If the distribution is symmetric about a value μ then the expected value equals μ.

$$\mathbb{E}X = \mu_X$$

variance is the measure of the variability or spread in a set of data. It can be defined as: '

$$var\left(X\right) = \mathbb{E}\left(\left(X - \mu_X\right)^2\right)$$
$$var\left(X\right) = \frac{\sum\left(X_i - \bar{X}\right)^2}{N}$$

Co-variance is a measure of the extent to which corresponding elements from two sets of ordered data move in the same direction. Co-variance between random variables X and Y, with

expected values μ_X and μ_Y, is defined as:

$$cov\left(X,Y\right) = \mathbb{E}\left(\left(X-\mu_X\right)\left(Y-\mu_Y\right)\right)$$
$$cov\left(X,Y\right) = \frac{\sum\left(X_i-\bar{X}\right)\left(Y_i-\bar{Y}\right)}{N}$$

Actually covariance measures how much two random variables vary together. Its similar to variance, but where variance tells you how a single variable varies, co variance tells you how two variables vary together.

3.6 The Chapter at a Glance

This chapter gives some background knowledge that is required for understanding PCA. This thesis includes some theoretical proofs that requires these mathematical knowledge. The next chapters provide the Streaming Online PCA algorithms and their theoretical, experimental results.

Chapter 4

Streaming Online PCA Algorithm

4.1 Introduction

Two online PCA algorithms are suggested in the paper [8]. These algorithms also have some limitations. The *Fixed Error: Conceptual Algorithm* uses incremental block size. As a result, at the end of iteration the incremental block becomes the input matrix X. So for a huge dataset X, this approach is not efficient. The *Fixed Error: Space Efficient Algorithm* uses covariance sketching . So it adds extra error ρ due to using covariance matrix sketching. Based on the limitations of these algorithms this chapter provides streaming online PCA algorithm that uses streaming buffer to predict the best principal components.

The algorithm (Algorithm 1) presented in this chapter is very simple. It uses the idea of streaming block buffer instead of using incremental sized block buffer or any covariance sketching algorithms like the algorithms presented in the paper [8]. The Algorithm 1 takes Δ that represents the fixed error bound and block size *b* which represents the history size of each streaming block as input. Each time *t*, the input vector $x_t \epsilon \mathbb{R}^d$ is given input to the algorithm and the corresponding output vector $y_t \epsilon \mathbb{R}^k$ is returned by the Algorithm 1 before taking input x_{t+1}. Algorithm 1 always guarantees that $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 < \Delta$ holds.

4.2 Flow Chart

The flow chart of this algorithm describing how the algorithm works is given below:

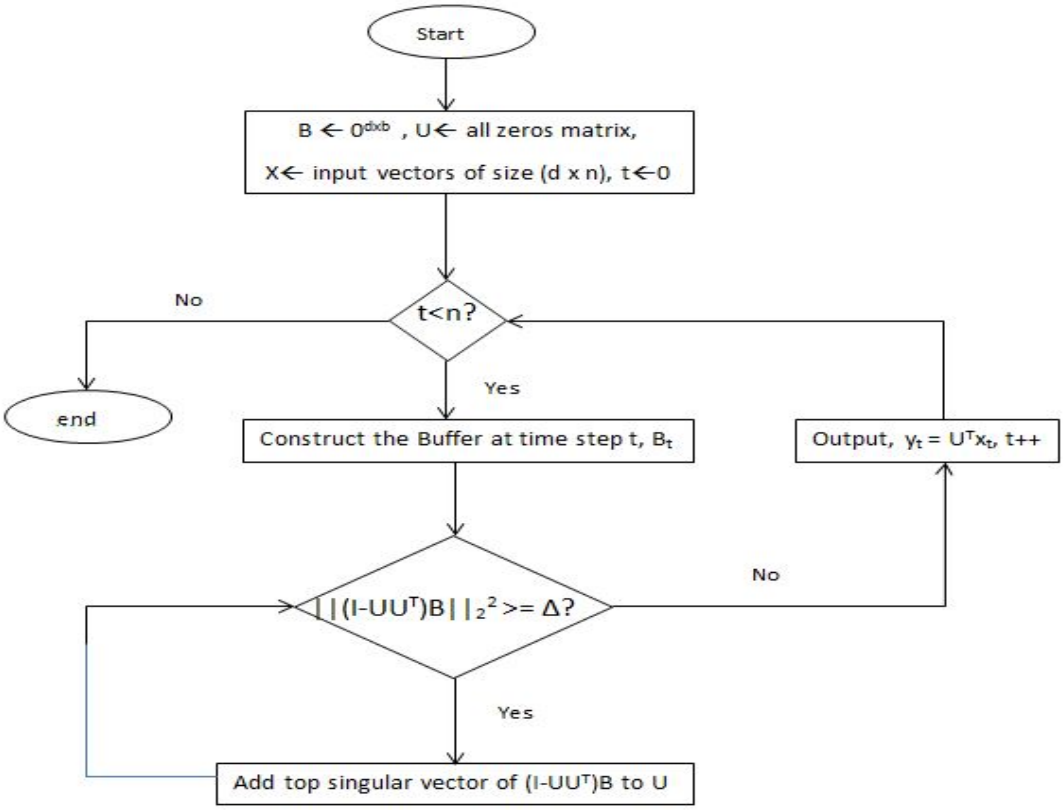

Figure 4.1: Flow chart of Algorithm 1

4.3 The Algorithm

At first, Algorithm 1 initializes the buffer $B \epsilon \mathbb{R}^{d \times b}$ and U with zeros. Then all the *n* input vectors are passed to the Algorithm 1 one by one. At time *t*, the block B_t contains the input vectors

$$B_t = [x_{t-b+1}, x_{t-b+2}, \ldots, x_{t-1}, x_t] \, \epsilon \mathbb{R}^{d \times b}$$

At first, Algorithm 1 checks whether or not

$$\left\|(I - U_t U_t^T) B_t)\right\|_2^2 < \Delta$$

If the algorithm finds that this is true, then it just computes the corresponding output vector y_t with respect to the current U_t. Otherwise if $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 \geq \Delta$ then the Algorithm 1 enters into the while loop and performs singular value decomposition of $(I - U_t U_t^T) B_t$. Then it adds the top singular vector to the next non-zero column of U_t. So the U_t is updated and this iteration continues until $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 < \Delta$ becomes true. When Algorithm 1 exits the while loop, it calculates the corresponding output vector y_t. The output dimension of y_t at time t depends on the number of left singular vectors i.e. total no of columns in U_t. So the output dimension of vectors varies at different times. At the end of this algorithm, total number of columns of U_n represents the final dimension of reduced output vectors.

Algorithm 1 Streaming Online PCA algorithm

Input:
X; input vectors, matrix of size $d \times n$
Δ ; the fixed error bound
b ; the streaming block size

$B \leftarrow 0^{d \times b}$
$U \leftarrow$ all zeros matrix
for $x_t \epsilon X$ **do**
  add x_t to the next zero column of B
  if B doesn't have any zero columns left **then**
    pop the 1st vector and push x_t at the end of B
  end if
  while $\left\|\left(I - UU^T\right) B\right\|_2^2 \geq \Delta$ **do**
    add top left singular vector of $\left(I - UU^T\right) B$ to U
  end while
  $y_t \leftarrow U^T x_t$
end for

4.4 Notations

In order to prove our results from Algorithm 1, we introduce some notations. x_t and y_t represent the input and output vector of dimension d, l respectively at time t. u_i represents the i-th column of projection matrix U and t_i represents the insertion time of u_i into U. σ_i denotes the i-th singular value of X in descending magnitude order. Also, U_t and B_t represents the state of matrix U and B at time t. At the end, the algorithm will return U_n and the number of columns of U_n represents the final dimension of reduced output vectors.

4.5 Findings

Based on the experimental and theoretical analysis, the observational and theoretical findings are given below:

4.5.1 Observations

Observation 1. *If we keep Δ fixed and gradually increase the streaming block size b, then the buffer B keeps track of more history than before. Consequently, $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ has high possibility to be greater than Δ. So, the possibility of going inside the while loop increases. As a result, the final dimension of output vector (l) increases and the Spectral error decreases.*

In Figure 6.3, we show the experimental proof of the Observation 1. The Figure 6.3 shows

that when we keep the fixed error bound Δ fixed and increase the streaming buffer size b, then $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2$ has high possibility to be greater than Δ. As a result the algorithm enters into the while loop and performs Singular Value Decomposition. Each time the algorithm enters into the while loop it adds the best left singular vector to the projection matrix. So the reduced output dimension l increases and spectral error decreases.

Observation 2. *If we keep the streaming block size b fixed and gradually increase the fixed error bound Δ. Then the condition $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ has low possibility to be true. As a result, the dimension of output vector (l) decreases and the Spectral error increases.*

Figure 6.4 illustrates the proof of our Observation 2. In this case we tune the value of fixed error bound Δ while keeping the buffer size b fixed. With the increase of Δ, the possibility of the condition $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ to be true becomes low. As a result the algorithm enters into the while loop less frequently resulting in a reduced output dimension with high reconstruction error.

Observation 3. *As we exit from the while loop when $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 < \Delta$. So each time t, the condition*

$$\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 < \Delta$$

is true.

This Streaming Online PCA Algorithm guarantees all the time that the condition

$$\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2$$

is always less than Δ. As we build up the algorithm in such a way that will check this condition before making projections of the input vectors. If the condition becomes false then it performs SVD on $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2$ and add the top principal vector on the projection vector. The algorithm iterates through the process until the condition becomes true. So we can say that Algorithm 1 always guarantees that the condition $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 < \Delta$ is always true.

Observation 4. *In order to be a true PCA algorithm, the projection matrix $\varphi = U$ must be an isometric matrix. In this Algorithm 1, all the time we add the top left singular vectors to U which are orthogonal with each other. So at the end of execution of the algorithm, the final projection matrix $\varphi = U_n$ we have, is an isometric matrix [30].*

we know that, an isometry is a transformation that preserves distance of the elements in the original metric space and maps elements to the same or another metric space. A matrix Q is called an orthogonal matrix if the following conditions hold:

1. $QQ^T = I$
2. $Q^T = Q^{-1}$

4.5.2 Relation between Isometry Matrix and Orthogonal Matrix

Let us consider the matrix A represents linear transformation that is an isometric matrix.

$$T(x) = Ax$$

Now,

$$\begin{aligned}\langle T(x), T(y)\rangle &= \langle Ax, Ay\rangle \\ &= (Ax)^T (Ay) \\ &= x^T A^T A y\end{aligned}$$

Here we consider matrix A represents linear transformation. So, from the properties of linear transformation we know that

$$\langle T(x), T(y)\rangle = \langle x, y\rangle$$

Finally we can write that, $A^T A = I$ i.e. A is orthogonal matrix.

4.5.3 Time and Space Complexity

Time Complexity describes how much time the algorithm consumes to run as a function of the length of the input. space complexity deals with quantifying how much (extra)space would be required by the algorithm to run as a function of the length of the input. Time and space complexity depends on lots of things like hardware, operating system, processors, datastructure used by the algorithms etc. But, we don't consider these factors at the time of analyzing the algorithm.
The time and space complexity of this algorithm is given below:

Lemma 1. (Time Complexity) *Algorithm 1 requires* $O(n \cdot min(d, b) \cdot min(db^2, bd^2))$ *arithmetic operations.*

Proof. The for loop at line number 3 iterates *n* times where *n* represents the number of input vectors each of dimension *d*. Here the dimension of $\left(I - U_t U_t^T\right) B_t$ is $d \times b$. So the while loop at line number 8 iterates maximum $min(d, b)$ times. To add the top left singular vector of $\left(I - U_t U_t^T\right) B_t$ to U at line number 9, the algorithm has to perform SVD that also requires $min(db^2, bd^2)$ operations.

Finally, we conclude that the Algorithm 1 requires

$$O\left(n \cdot min(d, b) \cdot min\left(db^2, bd^2\right)\right)$$

arithmetic operations. □

Lemma 2. (Space Complexity) *Algorithm 1 requires* $O\left(max\left(db, dl\right)\right)$ *space.*

Proof. In Algorithm 1, the buffer size B is $d \times b$. At the end of execution of Algorithm 1, the size of projection matrix (U) is at most $d \times l$. So Algorithm 1 requires at most $O\left(max\left(db, dl\right)\right)$ space. □

4.5.4 Bound on Output Dimension

The reduced output dimension is not fixed in this algorithm but here we are giving an upper bound on the output dimension l.

Theorem 1. *Let,* $\sigma_{k+1}^2 < \Delta$ *where* $k < l$ *and* σ_i *represent the singular values of X sorted in descending magnitude order. Then,*

$$l \leq \frac{k\left(\sigma_1^2 - \sigma_{k+1}^2\right)}{\Delta - \sigma_{k+1}^2}$$

Proof. Let at time $t = t_i - 1$,

$$\left\|\left(I - U_{t_i-1}U_{t_i-1}^T\right) B_{t_i-1}\right\|_2^2 < \Delta$$

and at time $t = t_i$, $\left\|\left(I - U_{t_i-1}U_{t_i-1}^T\right) B_{t_i}\right\|_2^2 < \Delta$ becomes false and u_i is inserted into U_{t_i-1}. Here u_i represents the topmost left singular vector of $\left(I - U_{t_i-1}U_{t_i-1}^T\right) B_{t_i}$.

$$\begin{aligned}
\Delta &\leq \left\|\left(I - U_{t_i-1}U_{t_i-1}^T\right) B_{t_i}\right\|_2^2 \\
&= sup\left\|u_i^T\left(I - U_{t_i-1}U_{t_i-1}^T\right) B_{t_i}\right\|_2^2 \\
&\leq \left\|u_i^T B_{t_i}\right\|_2^2
\end{aligned}$$

Now the total error is,

$$\begin{aligned}
l\Delta &\leq \sum_{i=1}^{l}\left\|u_i^T B_{t_i}\right\|_2^2 \\
&\leq \sum_{i=1}^{l}\sigma_i^2 \\
&\leq k\sigma_1^2 + (l-k)\sigma_{k+1}^2
\end{aligned}$$

Finally we have,

$$l \leq \frac{k\left(\sigma_1^2 - \sigma_{k+1}^2\right)}{\Delta - \sigma_{k+1}^2}$$

□

The output dimension bound of this algorithm is better than the bound of *Fixed Error: Space Efficient Algorithm* presented in [8]. As we don't use any sketching algorithm here so our algorithm is free from sketching error ρ.

4.6 The Chapter at a Glance

This chapter shows our first Online Streaming PCA Algorithm that considers the streaming buffer block concept for online version of PCA. Instead of using any kind of sketching algorithms our Algorithm 1 considers the buffer concept, as a result it can avoid the sketching error. The main limitation of this algorithm is that for this Algorithm 1 we can't give say the exact dimension of reduced output vector. Although we provide an upper bound of this dimension but it is just a bound. To avoid this problem in the next chapter we propose our second algorithm *Streaming Online PCA with Fixed Output Dimension* that avoids this problem by explicitly taking input the reduced output dimension. As a result the Algorithm 2 doesn't require any kind of bound on output dimension.

Chapter 5

Streaming Online PCA with Fixed Output Dimension

5.1 Introduction

In the Streaming Online PCA Algorithm 1, we give an upper bound on the dimension of output vector as the output dimension varies at different time. Our second algorithm (Algorithm 2) takes an extra parameter l as input, the fixed dimension of output vectors. It also maintains an array σ_arr of size l that keeps track of the best l top singular values. Our second algorithm makes the dimension of output vector l fixed but it can't guarantee $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 < \Delta$ all the time.

5.2 The Algorithm

The input vectors x_t each of dimension d is passed into the algorithm (Algorithm 2) one by one. Then the streaming block is formed like Algorithm 1. When $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 \geq \Delta$, Algorithm 2 enters into the while loop. It then calculates the top singular values and top left singular vectors of $\left(I - U_t U_t^T\right) B_t$. When top singular value is greater than at least one value in σ_arr, then Algorithm 2 replaces that minimum singular value of σ_arr and the corresponding column vector of U_t with the top singular value and the top left singular vector respectively. If the top singular value is not greater than any other value of σ_arr, then Algorithm 2 exits from the while loop and calculates the corresponding output y_t. At the end of the execution of Algorithm 2, we have the topmost l singular values in σ_arr and the best l left singular vectors in U_n.

Algorithm 2 Streaming Online PCA algorithm with fixed output dimension

Input:

X ; input vectors matrix of size $d \times n$
Δ ; the fixed error bound
b ; the streaming block size
l ; the fixed output vectors dimension

$B \leftarrow O^{d \times b}$
$U \leftarrow O^{d \times l}$
$\sigma_arr \leftarrow 0^{1 \times l}$
for $x_t \epsilon X$ **do**
    add x_t to the next zero column of B
    if B doesn't have any zero columns left **then**
        pop the 1st vector and push x_t at the end of B
    end if
    while $\left\| \left(I - UU^T\right) B \right\|_2^2 \geq \Delta$ **do**
        $[\sigma, u] \leftarrow$ top singular value and left singular vector of $\left(I - UU^T\right) B$
        if σ is less than all σ values of σ_arr **then**
            break
        else
            replace the minimum value of σ_arr with σ and also replace the corresponding vector from U with u
        end if
    end while
    $y_t \leftarrow U^T x_t$
end for

5.3 Notations

In order to prove our results from Algorithm 2, we introduce some notations. Here, x_t and y_t represent the input and output vector of dimension d, l respectively. B represents the streaming buffer of size $d \times b$. U represents the projection matrix φ of size $d \times l$. σarr represents the buffer that keeps track of the best l singular values. σ and u represents the top singular value of $\left(I - UU^T\right) B$.

5.4 Flow Chart

The flow chart of this algorithm is given below:

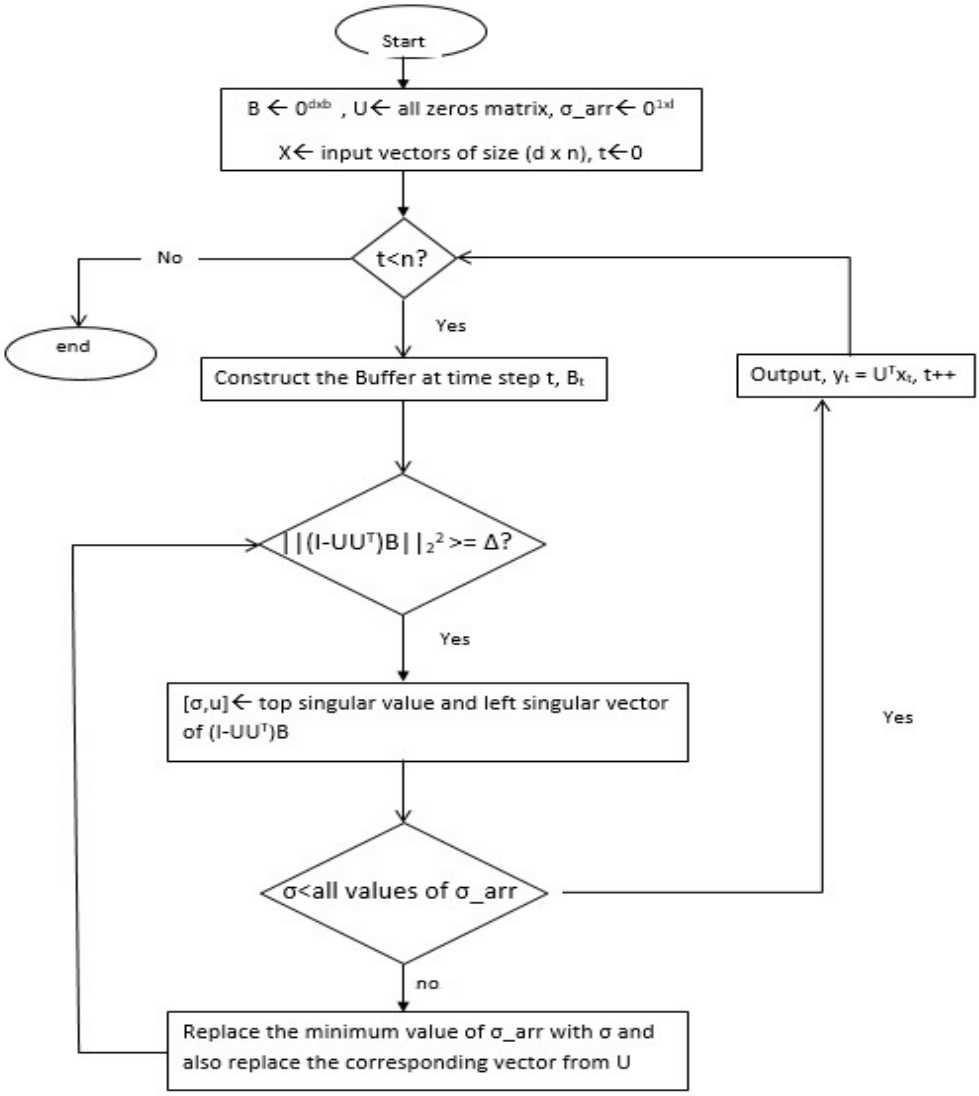

Figure 5.1: Flow chart of Algorithm 2

5.5 Findings

The main advantage of this Algorithm 2 is that it takes input l as input and it maintains an extra array σarr to keep track of the best singular values so that the projection matrix contains the best principal components. The observations and theoretical findings are explained below:

5.5.1 Observations

The observational findings from our experimental results are mentioned below:

Observation 5. *If we increase the dimension of output vectors l and keep the block size b and the fixed error bound Δ fixed in Algorithm 2, then the Spectral error decreases.*

In Figure 6.5 we show the relation between changes in output dimension l and the spectral reconstruction error $\|X - \varphi Y\|_2^2$. If we take high output dimension value for l, then the buffer σarr can keep track of more history than before. As a result, the projection matrix $\varphi = U$ contains the best l principal vectors resulting in a reduced spectral error.

Observation 6. *If we keep b , l fixed and decrease Δ in Algorithm 2, then the spectral error decreases.*

The Figure 6.6 illustrates the relationship between the spectral norm error $\|X - \varphi Y\|_2^2$ and the fixed error bound Δ. With the decrease in fixed tolerated error bound Δ, the spectral error decreases. Because with the decrease of Δ, the possibility of the condition $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 \geq \Delta$ to be true becomes low. As a result the algorithm enters into the while loop and search for a best principal components. If the singular value for this component is greater than any one singular values of σarr then the algorithm replaces the principal vectors of U corresponding to the lowers singular value with this new one. As a result, the spectral error decreases.

Observation 7. *If we keep Δ , l fixed and decrease the block size b in Algorithm 2, then the output spectral error increases.*

Figure 6.7 explains the relation between the streaming buffer size b and the spectral error $\|X - \varphi Y\|_2^2$. With the increase in block size b the buffer B can contain more previous input vectors than before. As a result, the possibility of the condition $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 \geq \Delta$ to be true becomes high and the algorithm enters into the while loop. Inside the while loop the algorithm explores more best principal components. Finally the total spectral error decreases.

5.5.2 Time and Space Complexity

The time and space complexity of this algorithm is given below:

Lemma 3. (Time and Space Complexity) *The Streaming Online PCA algorithm with fixed output dimension requires at most $O\left(n \cdot min\left(d, b\right) \cdot min\left(db^2, bd^2\right)\right)$ arithmetic operations and maximum $O\left(max\left(db, dl\right)\right)$ space.*

Proof. From Algorithm 2, we see that the for loop at line number 4 iterates n times. The while loop at line number 9 iterates at most $min\left(d, b\right)$ times. The complexity of performing SVD on $\left(I - U_t U_t^T\right) B_t$ at line number 10 to find out the top singular value and left singular vector is $min\left(db^2, bd^2\right)$.

Finally, we conclude that the Algorithm 2 requires

$$O\left(n \cdot min\left(d, b\right) \cdot min\left(db^2, bd^2\right)\right)$$

arithmetic operations.

In Algorithm 2, the buffer size B is $d \times b$. The size of projection matrix U is at most $d \times l$. So Algorithm 2 requires at most $O\left(max\left(db, dl\right)\right)$ space. $\square$

5.6 The Chapter at a Glance

In this Chapter we show Online Streaming PCA Algorithm with Fixed Output Dimension. Although this algorithm does not guarantee all the time that $\left\|(I - U_t U_t^T) B_t)\right\|_2^2 < \Delta$. The approach in which it computes the principal components ensures that we have the best approximation of principal components at that time step t. At the end of this algortihm execution, the projection matrix $\varphi = U$ contains the best l principal components. The next chapter will give the experimental section and the experimental proof of our findings till this point.

Chapter 6

Experimental Results

In this chapter, we compare our proposed algorithms with other online PCA mentioned in the paper [8]. We also experimentally show that our experimental results have matched with our findings described in chapter 1 and 2.

6.1 Experimental Setup

We perform our experimentaion in MATLAB environment. We use the $stocks$ matrix from $stockreturns$ dataset. $stocks$ is a 100×10 matrix containing double values. The double values in $stocks$ data matrix ranges from -5.87 to 6.46 . This dataset contains both positive and negative data.

6.2 Comparing Outputs between Different Algorithms

We compare our proposed Streaming Online PCA Algorithm 1 with two algorithms described in the paper [8]. The first one is *Fixed Error: Conceptual Algorithm* (CA) and the second one is *Fixed Error: Space Efficient Algorithm* (SEA).

6.2.1 Comparison based on Reconstruction Error

In Figure 6.1 we compare the outputs of our Streaming Online PCA Algorithm with the online PCA algorithms provided in the paper [8] named *Fixed Error: Conceptual Algorithm* (CA) and the second one is *Fixed Error: Space Efficient Algorithm* (SEA).

From the Figure 6.1, it is clear that the reconstruction error provided by our Algorithm 1 is better than the *Fixed Error: Space Efficient Algorithm* errors. Sometimes the reconstruction

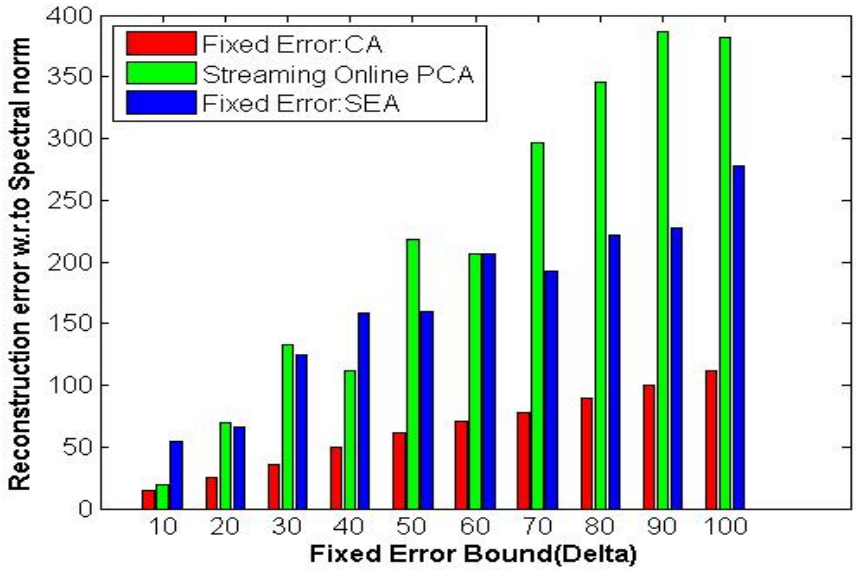

Figure 6.1: Reconstruction error vs Delta

error provided by our Algorithm 1 is greater than the error of this SEA but during that time the output diemnsion provided by our Algorithm 1 is lower than the output dimension of SEA. Here the *Fixed Error: Conceptual Algorithm* (CA) is providing the optimal best results as this algorithm considers online PCA using incremental fashion. But it is clear from the Figure 6.1 that our reconstruction error is close to the reconstruction error provided by the *Fixed Error: Conceptual Algorithm*.

6.2.2 Comparison based on Reduced Output Dimension

In Figure 6.2, we illustrate the comparison between reduced output dimension l of our Algorithm 1 and *Fixed Error: Conceptual Algorithm* (CA) and the second one is *Fixed Error: Space Efficient Algorithm* (SEA) proposed in the paper [8].

From the Figure 6.2 it is clear that reduced output dimension of our algorithm is closer, sometimes less than the *Fixed Error: Space Efficient Algorithm* (SEA) and always less than the *Fixed Error: Conceptual Algorithm* (CA).

When our output dimension is less than the *Fixed Error: Space Efficient Algorithm* (SEA) then the coresponding reconstruction error is greater then SEA. Also when our reduced output dimension is greater than the SEA then the spectral reconstruction error is less than SEA. So we can say that both cases our algorithm outperforms the algorithms presented in other papers.

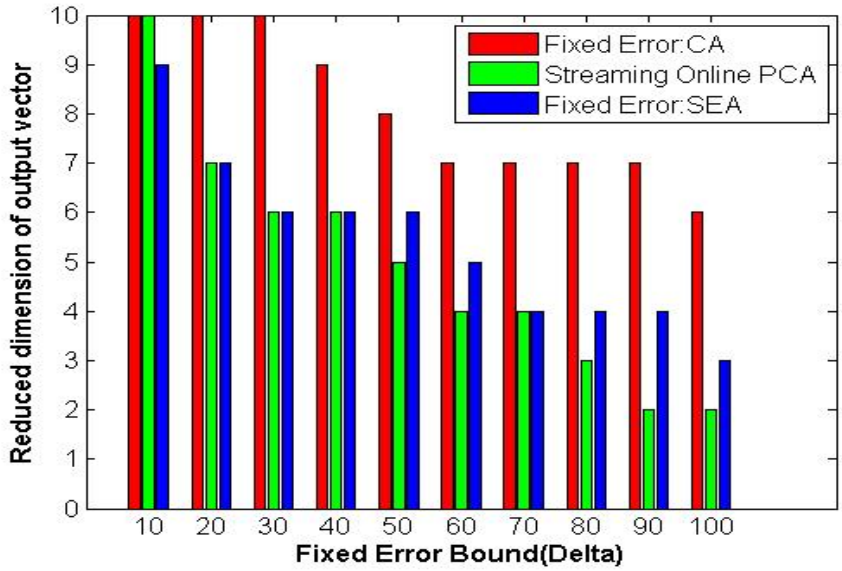

Figure 6.2: output dimension l vs Delta

6.3 Streaming Online PCA Algorithm and Observations

In this section, we show the experimental results of different observations we have already discussed in our chapter 4.

6.3.1 Reconstruction Error vs Block Size Tuning

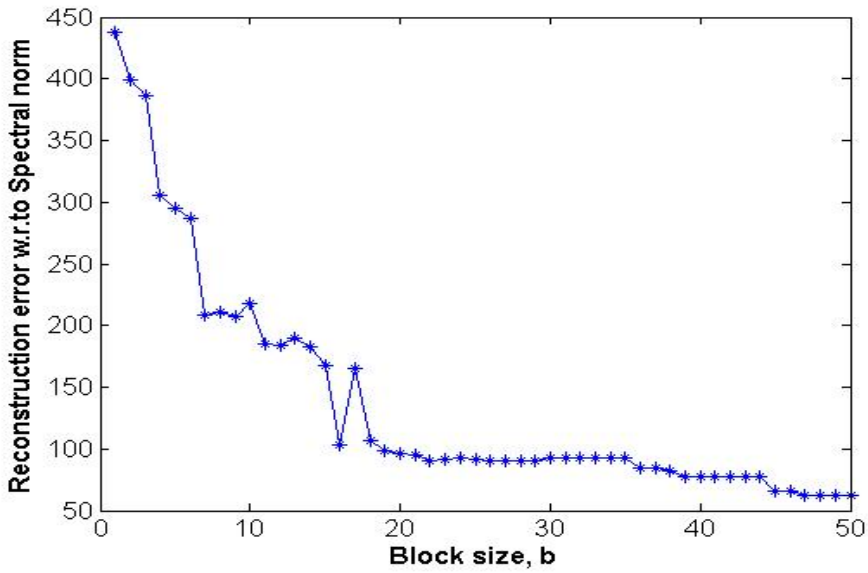

Figure 6.3: Impact on reconstruction error with the change in block size

In the Figure 6.3, we show that with the increase of streaming block size b the reconstruction error $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ decreases, where all the other parameters are kept constant. As

with the increase of block size b the Streaming Online PCA Algorithm 1 can keep history of more previous input vectors. As a result, the possibility of $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ to be greater than Δ increases. Consequently, the corresponding reconstruction error minimizes with the increase in block size b.

6.3.2 Change in Reconstruction Error with the Change in Fixed Error Bound

In the Figure 6.4, we show the relationship between spectral error and fixed error bound Δ. We see that with the increase in fixed error bound Δ and keeping the streaming buffer size b fixed, the corresponding dimension of output vector l decreases and the spectral error $\|X - \varphi Y\|_2^2$ increases.

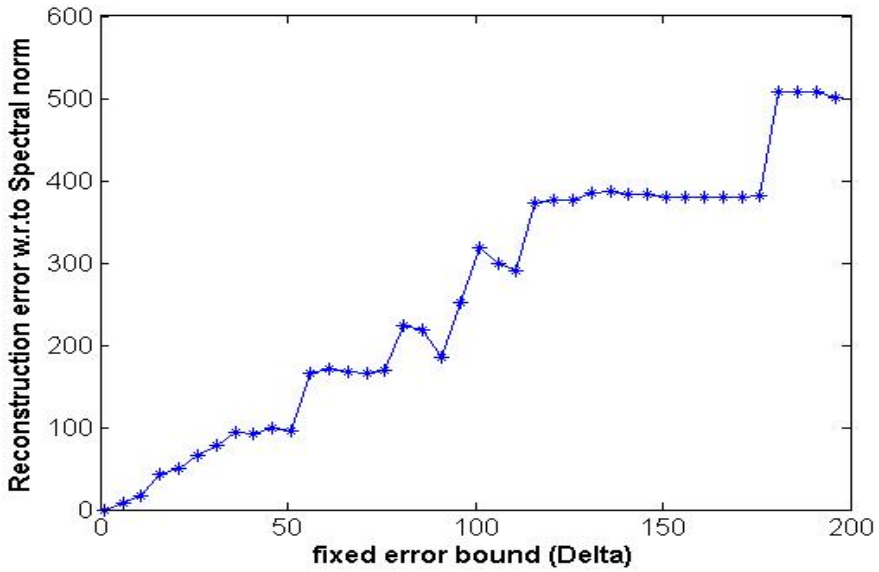

Figure 6.4: Relation between reconstruction error and fixed error bound

6.4 Streaming Online PCA with Fixed Output Dimension and Observations

In Chapter 5, we propose our second algorithm *Streaming Online PCA with Fixed Output Dimension*. The main difference of this algorithm with the Algorithm 1 is that it takes the reduced output dimension l as input with the other parameter of Algorithm 1. Together with the streaming buffer B, this algorithm also maintains an extra buffer σarr that keeps track of the best l singular values corresponding to the principal vectors in the projection matrix. This experimentation also done in MATLAB where the $stocks$ matrix of size 100×10 is used from the

$stockreturns$ dataset.

6.4.1 Changes in Reconstruction Error with the Change in Output Vectors Dimension

In Figure 6.5, we show the relation between spectral reconstruction error and reduced dimension of output vectors. If we keep the block size b and fixed error bound Δ fixed and increase the output dimension l in Algorithm 2, then the spectral reconstruction error $\|X - \varphi Y\|_2^2$ decreases. As the less reduction in output dimension the more principal components we can consider in our projection matrix. As a result, the spectral error becomes small.

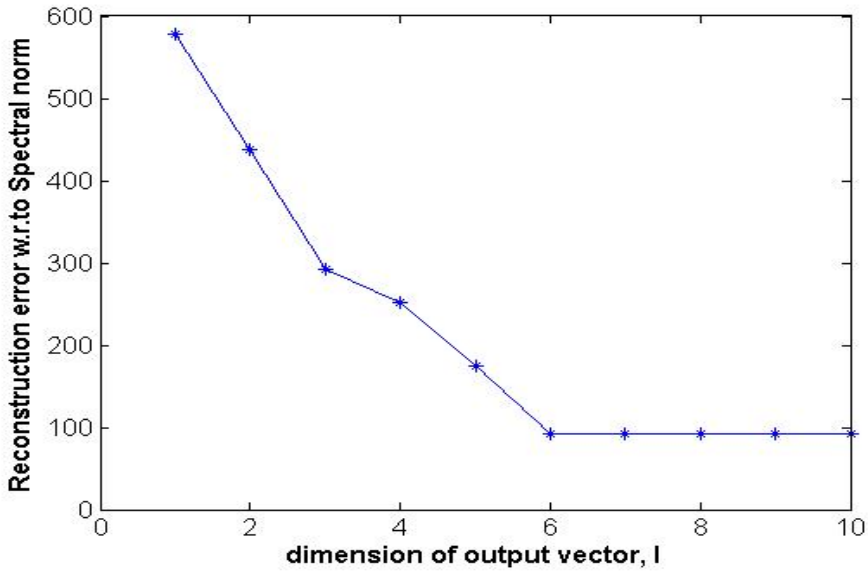

Figure 6.5: Relation between reconstruction error and output dimension as described in Observation 5

6.4.2 Changes in Reconstruction Error with the Change in Fixed Error Bound

The Figure 6.6 illustrates the effect on spectral reconstruction error $\|X - \varphi Y\|_2^2$ when the fixed error bound Δ is changed. From the Figure 6.6, it is clear that with the increase of fixed error bound Δ while keeping the block size b and output dimension l fixed, the reconstruction error increases. As with the increase in Δ the possibility of the condition $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2 \geq \Delta$ to be true becomes low. As a result the reconstruction error increases.

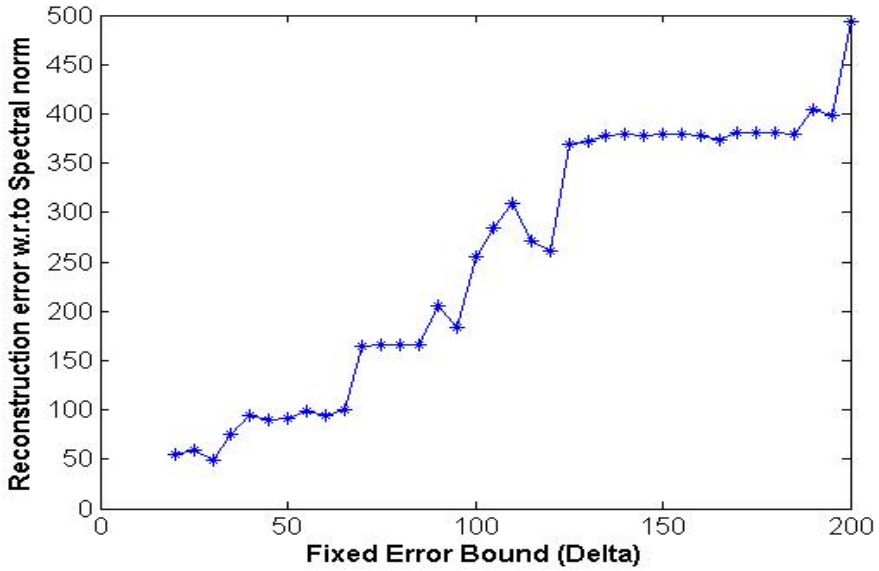

Figure 6.6: Observation 6 experimental results

6.4.3 Effect of Change in Buffer Size, b

The following Figure 6.7 shows the effect on spectral error with the change in buffer size b. The figure shows that with the increase in buffer size b the spectral error $\|X - \varphi Y\|_2^2$ decreases.

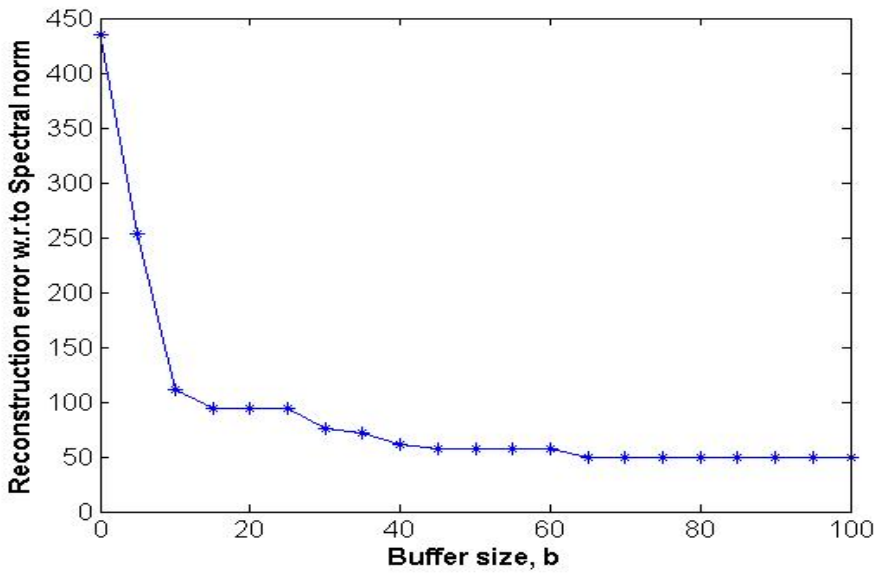

Figure 6.7: Observation 7 experimental results

With the increase in buffer size b the buffer B can maintain more history of inputs than before. As a result, the probability of $\left\|\left(I - U_t U_t^T\right) B_t\right\|_2^2$ to be greater than Δ becomes high. So the algorithm can generate more and more principal components and updates the σarr more frequently. As a result there exists high possibility that the projection matrix U contains the

principal components that is closer to the principal components obtained in offline standard PCA case.

6.5 The Chapter at a Glance

In this chapter, we illustrate the experimental sections. Also we compare the outputs of our algorithms with other standard online PCA algorithms. This chapter also provide experimental proof of different observation we established so far in Chapter 4 and 5.

Chapter 7

Conclusion

There exists different versions of online PCA algorithms considering different techniques. In this thesis, we consider streaming buffer block concept in online PCA and present two algorithms. In Algorithm 1, we establish a fixed error bound model and provide a Spectral bound on error. In Algorithm 2, we present fixed output dimension model where the dimension of output vectors remains constant. Both theoretically and experimentally, we show that our algorithms generate better results and better reconstruction error in comparing with the other online PCA algorithms.

7.1 Future Works

In this thesis, we present algorithms using the concept of streaming block buffer in online PCA. In future, we would like to handle the fixed error bound Δ automatically inside the algorithm to get the optimal results instead of giving input Δ to the algorithm directly. In this setting of our algorithm we have considered the spectral norm error and we also give a spectral bound. We would like to apply this concept in Frobenius norm case. Another possibility is to use this streaming online block concept with offline standard PCA algorithm to solve the online PCA problem.

References

[1] "Principal component analysis." https://en.wikipedia.org/wiki/Principal_component_analysis.

[2] G. H. Dunteman, "Principal component analysis (quantitative applications in the social sciences)," 1989.

[3] L. I. Smith, "A tutorial on principal components analysis," *Cornell University, USA*, vol. 51, p. 52, 2002.

[4] "Principal components analysis." http://www.di.fc.ul.pt/~jpn/r/pca/pca.html.

[5] K. Baker, "Singular value decomposition tutorial," *The Ohio State University*, vol. 24, 2005.

[6] M. Holmes, A. Gray, and C. Isbell, "Fast svd for large-scale matrices," in *Workshop on Efficient Machine Learning at NIPS*, vol. 58, pp. 249–252, 2007.

[7] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.

[8] Z. Karnin and E. Liberty, "Online pca with spectral bounds," in *Proceedings of the 28th Annual Conference on Computational Learning Theory (COLT)*, pp. 505–509, 2015.

[9] C. Musco and C. Musco, "Randomized block krylov methods for stronger and faster approximate singular value decomposition," in *Advances in Neural Information Processing Systems*, pp. 1396–1404, 2015.

[10] P. Drineas and A. Zouzias, "A note on element-wise matrix sparsification via a matrix-valued bernstein inequality," *Information Processing Letters*, vol. 111, no. 8, pp. 385–389, 2011.

[11] D. Achlioptas and F. McSherry, "Fast computation of low-rank matrix approximations," *Journal of the ACM (JACM)*, vol. 54, no. 2, p. 9, 2007.

[12] S. Arora, E. Hazan, and S. Kale, "A fast random sampling algorithm for sparsifying matrices," in *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, pp. 272–279, Springer, 2006.

[13] E. Liberty, F. Woolfe, P.-G. Martinsson, V. Rokhlin, and M. Tygert, "Randomized algorithms for the low-rank approximation of matrices," *Proceedings of the National Academy of Sciences*, vol. 104, no. 51, pp. 20167–20172, 2007.

[14] T. K. Landauer, *Latent semantic analysis*. Wiley Online Library, 2006.

[15] T. Sarlos, "Improved approximation algorithms for large matrices via random projections," in *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*, pp. 143–152, IEEE, 2006.

[16] C. Boutsidis, P. Drineas, and M. Magdon-Ismail, "Near-optimal column-based matrix reconstruction," *SIAM Journal on Computing*, vol. 43, no. 2, pp. 687–717, 2014.

[17] C. Boutsidis, M. W. Mahoney, and P. Drineas, "An improved approximation algorithm for the column subset selection problem," in *Proceedings of the twentieth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 968–977, Society for Industrial and Applied Mathematics, 2009.

[18] M. K. Warmuth and D. Kuzmin, "Randomized online pca algorithms with regret bounds that are logarithmic in the dimension," *Journal of Machine Learning Research*, vol. 9, no. Oct, pp. 2287–2320, 2008.

[19] I. Mitliagkas, C. Caramanis, and P. Jain, "Memory limited, streaming pca," in *Advances in Neural Information Processing Systems*, pp. 2886–2894, 2013.

[20] C. Boutsidis, D. Garber, Z. Karnin, and E. Liberty, "Online principal components analysis," in *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 887–901, SIAM, 2015.

[21] E. Liberty, "Simple and deterministic matrix sketching," in *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 581–588, ACM, 2013.

[22] M. Ghashami and J. M. Phillips, "Relative errors for deterministic low-rank matrix approximations," in *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 707–717, Society for Industrial and Applied Mathematics, 2014.

[23] P. Drineas and R. Kannan, "Pass efficient algorithms for approximating large matrices.," in *SODA*, vol. 3, pp. 223–232, 2003.

[24] "Vector norms." http://fourier.eng.hmc.edu/e161/lectures/algebra/node11.html.

[25] "Matrix norms." http://fourier.eng.hmc.edu/e161/lectures/algebra/node12.html.

[26] "Eigenvalues and eigenvectors." http://www.math.harvard.edu/archive/20_spring_05/handouts/ch05_notes.pdf.

[27] "Singular value decomposition." http://lall.stanford.edu/svn/ee263_2010_to_2011_spring/data/svd_2011_04_19_01.pdf.

[28] J. M. Phillips, "Singular value decomposition." https://www.cs.utah.edu/~jeffp/teaching/cs5140-S15/cs5140/L15-SVD.pdf.

[29] "Singular value and eigenvalue decomposition." http://www.cc.gatech.edu/~dellaert/pub/svd-note.pdf.

[30] D. Cai, X. He, and J. Han, "Isometric projection," in *AAAI*, pp. 528–533, 2007.

Appendix A

Codes

A.1 Streaming Online PCA Algorithm

```
function [ Y,spect_err,col ] = Fully_Modified_Algo2( X,delta,k
    )
%Summary of this function goes here
%   Detailed explanation goes here
%   X=input matrix of dim d X n
%   k=block size
%       delta= acceptable tolerance limit
%   Y=output matrix of dim l X n
%   ------------------------------------------------------


[d,n]=size(X);

Y=zeros(d,n);        %just initializing the output

U=zeros(d,0);
B=zeros(d,k);
I=eye(d);

for t=1:n
    x=X(:,t);

    if(t>=1 && t<=k)
        B=[B(:,1:t-1),x,B(:,t+1:k)];
```

```
    else
        B=[B(:,2:k),x];
    end

    while(norm((I-U*U')*B)^2>=delta)
        [svd_U,svd_S,svd_V]=svds((I-U*U')*B,1);

        U=[U,svd_U];
    end

    [l,c]=size(U'*x);

    Y(:,t)=Y(:,t)+[U'*x;zeros(d-l,1)];

end

[a,b]=size(Y);


[row,col]=size(U);

spect_err=norm(X-U*Y(1:col,:))^2;
end
```

A.2 Streaming Online PCA with Fixed Output Dimension

```
function [ Y,spect_err] = Fully_Modified_Algo( X,k,l,delta )
%Summary of this function goes here
%   Detailed explanation goes here
%   X=input matrix of dim d X n
%   k=block size
%       delta= acceptable tolerance limit
%   Y=output matrix of dim l X n
%   ------------------------------------------------------------

[d,n]=size(X);
B=zeros(d,k);
U=zeros(d,l);
sigma_list=zeros(1,l);                  %keeps all the sigmas in
```

```
    square form

I=eye(d);
Y=zeros(l,0);

for t=1:n
    x=X(:,t);

    if(t>=1 && t<=k)
        B=[B(:,1:t-1),x,B(:,t+1:k)];
    else
        B=[B(:,2:k),x];
    end

    while(norm((I-U*U')*B)^2>=delta)

        [svd_U,svd_S,svd_V]=svds((I-U*U')*B,1);
        [minm,idx]=min(sigma_list(1,:));

        if(minm>=svd_S)
            break;
        else
            sigma_list=[sigma_list(1,1:idx-1),svd_S^2,sigma_list
                (1,idx+1:l)];
            U=[U(:,1:idx-1),svd_U,U(:,idx+1:l)];
        end

    end
    Y=[Y,U'*x];
end
spect_err=norm(X-U*Y)^2;
end
```

A.3 Observation 1 Experimental Code

```
load stockreturns;

X=stocks';

```

```
Myerror=zeros(1,0);

for i=1:50
    [ Y,spect_err,out_dim ] = Streaming_Online_PCA1( X,50,i );
    Myerror=[Myerror,spect_err];
end

x=[1:1:50];
y1=Myerror;

p=plot(x,y1,'b-*');
p.LineWidth=3;
set(gca, 'FontName', 'Arial');
set(gca, 'FontSize', 12);
xlabel('\bfBlock size, b'),ylabel('\bfReconstruction error w.r.
    to Spectral norm')
```

A.4 Observation 2 Experimental Code

```
load stockreturns;

X=stocks';

Myerror=zeros(1,0);

for i=1:5:200
    [ Y,spect_err,out_dim ] = Streaming_Online_PCA1( X,i,20 );
    Myerror=[Myerror,spect_err];
end

x=[1:5:200];
y1=Myerror;

p=plot(x,y1,'b-*');
p.LineWidth=3;
set(gca, 'FontName', 'Arial');
set(gca, 'FontSize', 12);
xlabel('\bffixed error bound (Delta)'),ylabel('\
    bfReconstruction error w.r.to Spectral norm')
```

A.5 Observation 5 Experimental Code

```
load stockreturns;

X=stocks';

Myerror=zeros(1,0);

for i=1:10
    [ Y,spect_err ] = Streaming_Online_PCA2( X,25,i,50 );
    Myerror=[Myerror,spect_err];
end

x=[1:1:10];
y1=Myerror;

p=plot(x,y1,'b-*');
p.LineWidth=3;
set(gca, 'FontName', 'Arial');
set(gca, 'FontSize', 12);
xlabel('\bfdimension of output vector, l'),ylabel('\
    bfReconstruction error w.r.to Spectral norm')
```

A.6 Observation 6 Experimental Code

```
load stockreturns;

X=stocks';

Myerror=zeros(1,0);

for i=20:5:200
    [ Y,spect_err ] = Streaming_Online_PCA2( X,25,8,i );
    Myerror=[Myerror,spect_err];
end

x=[20:5:200];
y1=Myerror;

```

```
p=plot(x,y1,'b-*');
p.LineWidth=3;
set(gca, 'FontName', 'Arial');
set(gca, 'FontSize', 12);
xlabel('\bfFixed Error Bound (Delta)'),ylabel('\
    bfReconstruction error w.r.to Spectral norm')
```

A.7 Observation 7 Experimental Code

```
load stockreturns;

X=stocks';

Myerror=zeros(1,0);

for i=0:5:100
    [ Y,spect_err ] = Streaming_Online_PCA2( X,i,8,40 );
    Myerror=[Myerror,spect_err];
end

x=[0:5:100];
y1=Myerror;

p=plot(x,y1,'b-*');
p.LineWidth=3;
set(gca, 'FontName', 'Arial');
set(gca, 'FontSize', 12);
xlabel('\bfBuffer size, b'),ylabel('\bfReconstruction error w.r
    .to Spectral norm')
```

Generated using Undegraduate Thesis LaTeX Template, Version 1.2. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Friday 17th February, 2017 at 5:32pm.